

Setup Supabase — Estudee

1. Criar projeto no Supabase

1. Acesse <https://supabase.com> e crie um novo projeto
2. Anote: **Project URL** e **Anon Key** (Settings > API)

2. Instalar dependências

```
bash

npm install @supabase/supabase-js @supabase/ssr
```

3. Variáveis de ambiente

Crie um arquivo `.env.local` na raiz do projeto:

```
env

NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
```

4. Configurar autenticação no Supabase

No painel do Supabase, vá em **Authentication** > **Providers** e certifique-se que o **Email** está habilitado.

Em **Authentication** > **URL Configuration**, configure:

- Site URL: `http://localhost:3000` (dev) / seu domínio em produção
- Redirect URLs: `http://localhost:3000/auth/callback`

5. Criar tabela de perfis

Execute no **SQL Editor** do Supabase:

```
sql
```

```
-- Tabela de perfis estendidos
create table public.profiles (
  id uuid references auth.users on delete cascade primary key,
  nome text not null,
  sobrenome text not null,
  cidade text not null,
  estado text not null,
  telefone text,
  concurso_alvo text,
  avatar_url text,
  plano text not null default 'gratuito', -- 'gratuito' | 'premium'
  pontuacao integer not null default 0,
  streak_dias integer not null default 0,
  streak_ultima_data date,
  created_at timestamp with time zone default now()
);

-- Habilitar RLS
alter table public.profiles enable row level security;

-- Políticas de acesso
create policy "Usuário vê o próprio perfil"
  on profiles for select
  using (auth.uid() = id);

create policy "Usuário atualiza o próprio perfil"
  on profiles for update
  using (auth.uid() = id);

create policy "Perfil criado no cadastro"
  on profiles for insert
  with check (auth.uid() = id);

-- Trigger: cria perfil automaticamente ao registrar
create or replace function public.handle_new_user()
returns trigger as $$
begin
  insert into public.profiles (id, nome, sobrenome, cidade, estado)
  values (
    new.id,
    new.raw_user_meta_data->>'nome',
    new.raw_user_meta_data->>'sobrenome',
    new.raw_user_meta_data->>'cidade',
    new.raw_user_meta_data->>'estado'
  );
  return new;
end;
```

```

end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();

```

6. Logs de acesso (requisito LGPD/Marco Civil)

```

sql

create table public.logs_acesso (
  id bigserial primary key,
  user_id uuid references auth.users on delete set null,
  acao text not null, -- 'login', 'logout', 'inicio_simulado', etc.
  ip_origem text,
  created_at timestamp with time zone default now()
);

alter table public.logs_acesso enable row level security;

-- Apenas o sistema insere (via service_role), usuário não lê/escreve

```

Estrutura de arquivos gerada

```

src/
├── lib/
│   ├── supabase/
│   │   ├── client.ts      ← cliente browser
│   │   ├── server.ts      ← cliente server-side
│   │   └── middleware.ts   ← refresh de sessão
│   └── types/
│       └── database.ts     ← tipos do banco
├── app/
│   ├── (auth)/
│   │   ├── layout.tsx     ← layout das telas de auth
│   │   ├── cadastro/
│   │   │   └── page.tsx
│   │   ├── login/
│   │   │   └── page.tsx
│   │   └── auth/
│   │       ├── callback/
│   │       └── route.ts    ← callback OAuth/email
│   └── middleware.ts       ← proteção de rotas

```